

Proyecto de clase · IoT Security

FitCore IoT

Sistema de Seguridad

Análisis del proyecto, estado del equipo y roadmap

NodeMCU ESP8266 · HC-SR501 PIR · Firebase RTDB · React Native

Generado: Mayo 2026

1 — Descripción general del proyecto

FitCore IoT Security es un sistema de detección de intrusos en tiempo real compuesto por hardware físico (NodeMCU ESP8266 + sensor PIR HC-SR501 + buzzer), un backend en Firebase Realtime Database y una app Android en React Native / Expo. Adicionalmente incluye tres dispositivos simulados en HTML/JS: smartwatch, Smart TV y asistente de voz via IFTTT.

Stack tecnológico

Capa	Tecnología
Firmware	C++ / Arduino IDE 2.x · Firebase_ESP_Client (Mobizt)
Sensor	HC-SR501 (PIR, movimiento infrarrojo)
Actuador	Buzzer activo
Backend	Firebase RTDB + Cloud Functions + FCM
App	React Native / Expo (Android)
Simulados	HTML/JS — Smartwatch, Smart TV, IFTTT + Google Assistant

Circuito físico

Componente	Pin NodeMCU	Notas
HC-SR501 VCC	VIN (5 V)	El PIR necesita 5 V para operar correctamente
HC-SR501 OUT	D1 (código activo)	Sin divisor de voltaje — pin OUT es 3.3 V compatible
HC-SR501 GND	GND	—
Buzzer +	D2 (código activo)	Buzzer activo, señal digital 3.3 V

Componente	Pin NodeMCU	Notas
Buzzer —	GND —	

⚠ Inconsistencia de documentación El README principal y los diagramas dicen PIR→D2 y Buzzer→D5, pero el código funcional (`CodigoArduino/`) usa D1 para el PIR y D2 para el buzzer. El hardware está cableado según el código, no según el README. La documentación necesita corregirse para reflejar el cableado real.

2 — Estado del código

Firmware activo: CodigoArduino/

Este es el firmware validado por M1. Ya funciona y escribe valores en Firebase RTDB. Estructura de archivos:

- `CodigoPrincipal.ino` — Lógica principal: lectura PIR cada 1200 ms, buzzer, escritura a Firebase
- `ConexionFirebase.h` — Configuración WiFi + Firebase (credenciales en local, no subidas al repo)
- `Buzzer.h` — Funciones de inicialización, activación y apagado del buzzer

✓ Lo que funciona correctamente
Lectura del PIR cada 1200 ms · Activación del buzzer al detectar movimiento · Escritura booleana a Firebase RTDB · Warmup de 30 s para estabilización del PIR · Reconexión WiFi implícita por la librería `Firebase_ESP_Client`

Lo que le falta al firmware actual (requerido en S3)

Funcionalidad	Estado actual	Requerido en S3
Máquina de estados	No existe	WARMUP → ARMED → ALERT → ALARM

Funcionalidad	Estado actual	Requerido en S3
Path Firebase	<code>/garage/motion</code>	<code>/sensor/movimiento</code>
Campo <code>estado</code>	No enviado	String: "armado" / "alerta" / "alarma"
Campo <code>alerta</code>	No enviado	Bool en <code>/sensor/alerta</code>
Historial de eventos	No implementado	POST a <code>/alertas/{pushId}</code>
Armar/desarmar remoto	No implementado	Leer <code>/config/armado</code> cada 2 s
Intervalo de lectura	1200 ms	100 ms (roadmap S2)

● **Bloqueo crítico para M3** El firmware escribe en `/garage/motion` pero M3 va a leer `/sensor/movimiento`. Mientras no se alinee este path, M3 no puede avanzar en su tarea de S2. Es un cambio de una sola línea en el firmware.

Firmware secundario: `firmware/fitcore_security/`

Este es un firmware alternativo que usa sensor ultrasónico HC-SR04 (descartado) + ArduinoJson + REST HTTP. Tiene implementada la máquina de estados completa (DISARMED → ARMED → ALERT → ALARM) y puede servir como referencia directa para M1 al implementar la máquina de estados en S3, adaptando la lógica del sensor ultrasónico a un simple `digitalRead()`.

3 — Schema Firebase RTDB

Schema objetivo (PIR v1.1)

```
/sensor/movimiento    ← bool   (true cuando el PIR detecta movimiento)

/sensor/estado        ← string ("calentando" | "desarmado" | "armado" | "alerta" |
"alarma")

/sensor/alerta        ← bool

/config/armado         ← bool   (la app escribe aquí para armar/desarmar)

/alertas/{pushId}     ← { tipo: string, timestamp: number }
```

 **firebase_schema.json desactualizado** El archivo `docs/firebase_schema.json` todavía tiene el campo `distancia_cm` del sensor HC-SR04 descartado. M2 debe actualizarlo para que refleje el schema PIR real antes de que M3 empiece a conectar la app.

Reglas de seguridad

Archivo	Cuándo usarlo	Acceso
<code>firebase_rules_dev.js</code> <code>on</code>	S1-S4 (desarrollo)	Lectura y escritura abierta — sin autenticación
<code>firebase_rules_prod.js</code> <code>son</code>	S6+ (cuando la app tenga login)	Requiere <code>auth != null</code> en todas las rutas

4 — Análisis del Roadmap

Estado real de cada miembro

M1 Adelantado — hardware funcionando, código necesita evolucionar

-  PIR conectado y leyendo correctamente
-  Buzzer activándose con detección
-  Datos llegando a Firebase RTDB en tiempo real
-  Warmup implementado (30 s)
-  Path en Firebase incorrecto (`/garage/motion` en lugar de `/sensor/movimiento`)
-  Sin máquina de estados (requerido para S3)
-  Sin lectura remota armar/desarmar

M2 Firebase creado (implícito), pendiente de documentar y compartir

-  Firebase RTDB activo (M1 ya escribe datos)
-  Credenciales no compartidas con M3 (bloquea inicio de M3)
-  `firebase_schema.json` desactualizado (tiene HC-SR04, no PIR)

M3 No ha empezado — bloqueado por credenciales y path de Firebase

-  Proyecto Expo no creado
-  Sin credenciales Firebase
-  Path a leer (`/sensor/movimiento`) aún no existe en la DB

Fase 1 — Fundación (S1–S2)

Tarea	Miembro	Estado
Repo GitHub creado con estructura y ramas	TODOS	 Completado

Tarea	Miembro	Estado
Kickoff y asignación de roles M1/M2/M3	TODOS	✅ Completado
Arduino IDE + board ESP8266 + librería instalados	M1	✅ Completado
HC-SR501 conectado, señal verificada en Serial Monitor	M1	✅ Completado
Firebase RTDB creado en modo test	M2	✅ Completado
Schema RTDB documentado en firebase_schema.json (v1.1 PIR)	M2	❌ Pendiente
Proyecto Expo creado + firebase instalado	M3	❌ Pendiente
Primer POST a Firebase con movimiento:true (path correcto)	M1	⚠️ Funciona pero path incorrecto
Credenciales compartidas con M3 vía .env	M2	❌ Pendiente
App lee /sensor/movimiento y lo muestra en pantalla	M3	❌ Pendiente

🎯 **Milestone 1 — Estado** M1 cumplió su parte técnica. El pipeline ESP8266→Firebase existe y funciona. Falta: corregir el path, que M2 comparta credenciales, y que M3 lea el dato. El Milestone 1 está al 60 % — se completa cuando los tres están conectados al mismo nodo de Firebase.

Fase 2 — Núcleo IoT (S3–S4)

Esta fase es la más exigente para M1. Requiere implementar la máquina de estados completa:



Tarea S3-S4	Miembro	Notas
Implementar máquina de estados en firmware	M1	Referencia: firmware/fitcore_security/fitcore_security.ino
Leer /config/armado cada 2 s	M1	Permite armar/desarmar desde la app
POST a /alertas/{pushId} en cada evento	M1 + M2	Firebase genera el pushId con POST
Indicador visual de estado en app (colores)	M3	Verde=armado, amarillo=alerta, rojo=alarma, gris=desarmado
Historial de eventos en app	M3	Lista ordenada por más reciente
Botón Armar/Desarmar en app	M3	Escribe en /config/armado
Prueba integrada, medir latencia (meta: < 2 s)	M1+M2	PIR → Firebase → app

🎯 Milestone 2 — Sistema completo base PIR detecta → estado actualizado en Firebase → app muestra estado en tiempo real → app puede armar/desarmar remotamente.

Latencia medida y documentada.

Fase 3 — App Android completa (S5–S6)

Tarea	Miembro
Habilitar Firebase Cloud Messaging (FCM), obtener Server Key	M2
Cloud Function: dispara FCM al crearse nueva alerta en /alertas/{id}	M2
Integrar expo-notifications en la app, guardar FCM token	M3
Dashboard completo: estado, última detección, contador del día	M3
Activar Firebase Auth + aplicar firebase_rules_prod.json	M2

🎯 **Milestone 3 — App Android completa** Dashboard, historial, notificaciones push y autenticación activa.

Fase 4 — Dispositivos simulados (S7–S8)

Dispositivo	Tecnología	Responsable
Smartwatch	HTML circular 250×250 px · Firebase SDK WebSocket	M3
Smart TV	HTML full-screen · REST polling cada 5 s	M3
Asistente de voz	IFTTT Webhook → Google Assistant broadcast	M2
Documentar circuito (foto + tabla de pines)	—	M1

🎯 **Milestone 4 — Los 4 dispositivos alertan correctamente** App Android, Smartwatch, Smart TV y asistente de voz responden al mismo evento.

Fase 5 — Pruebas (S9)

- Prueba end-to-end 5 veces: mano frente al PIR → todos los dispositivos alertan
- Documentar latencia real medida (meta: < 3 s)
- Resolver bugs críticos y hacer PR final a `main`
- Grabar video demo + Plan B para WiFi de clase

Fase 6 — Entrega (S10)

- README final por módulo: firmware (M1), backend (M2), app/simulados (M3)
- Repo limpio: `main` actualizado, `secrets.h` en `.gitignore`, sin credenciales
- Demo en vivo con hotspot de respaldo

5 — Prioridades inmediatas para desbloquear el equipo

#	Acción	Miembro	Impacto
1	Cambiar <code>/garage/motion</code> → <code>/sensor/movimiento</code> en <code>CodigoPrincipal.ino</code>	M1	Desbloquea a M3 — cambio de una línea
2	Compartir credenciales Firebase con M3 via <code>.env</code> (nunca al repo)	M2	M3 puede arrancar S2
3	Actualizar <code>docs/firebase_schema.json</code> al schema PIR v1.1	M2	Documentación alineada
4	M3 crea proyecto Expo + instala firebase + verifica lectura en RTDB	M3	Completa Milestone 1
5	Corregir tabla de pines en README (D1/D2 según M1		Evita confusión al cablear

#	Acción	Miembro	Impacto
	código real)		
6	M1 implementa máquina de estados para S3 (referencia: <code>firmware/fitcore_security/</code>)	M1	Habilita Milestone 2

6 — Resumen de issues del proyecto

Severidad	Issue	Archivo
 Crítico	Path Firebase incorrecto: <code>/garage/motion</code> vs <code>/sensor/movimiento</code>	<code>CodigoPrincipal.ino:46,59</code>
 Importante	README dice pines D2/D5 pero código usa D1/D2	<code>README.md + Buzzer.h:4 + CodigoPrincipal.ino:5</code>
 Importante	<code>firebase_schema.json</code> tiene <code>distancia_cm</code> de HC-SR04, no PIR	<code>docs/firebase_schema.json</code>
 Importante	Sin máquina de estados — requerida para S3	<code>CodigoPrincipal.ino</code>
 Menor	<code>ConexionFirebase.h</code> no está en <code>.gitignore</code>	<code>.gitignore</code>
 Menor	<code>calibracion.md</code> vacío	<code>docs/calibracion.md</code>
 Info	<code>app/</code> y <code>simulados/</code> aún no existen en el repo	—

Code